11 MAGGIO 2026

Analisi statica dei programmi / Ispezione del codice

MAFFEIS RICCARDO 1085706

Sommario

1. Introduzione 2
2. Metodo di ispezione 2
3. Riepilogo dei 20 punti di controllo 2
4. Analisi dettagliata punto per punto 4
1. Obiettivo dell'analisi 4
2. Struttura generale del programma 4
3. Leggibilità del codice 5
4. Correttezza logica 5
5. Verifica delle precondizioni 6
6. Verifica delle postcondizioni 6
7. Verifica degli invarianti 7
8. Controllo dei casi limite 7
9. Gestione degli errori 8
10. Controllo della sicurezza 8
11. Analisi delle dipendenze 9
12. Controllo della duplicazione del codice 9
13. Verifica della manutenibilità 10
14. Controllo della documentazione 10
15. Verifica dello stile di programmazione 11
16. Analisi preliminare delle prestazioni 11
17. Controllo della gestione delle risorse 12
18. Analisi della testabilità 12
19. Individuazione di codice morto o irraggiungibile 13
20. Esito finale dell'ispezione 13
5. Criticità individuate e miglioramenti consigliati 14
6. Punti di forza rilevati 14
7. Conclusione 15

1. Introduzione

Il presente documento raccoglie l'analisi statica e l'ispezione del codice del sistema ascensore. L'ispezione è stata eseguita senza considerare l'esecuzione del programma come criterio primario, ma analizzando struttura, contratti, flusso logico, gestione degli stati e coerenza rispetto ai requisiti funzionali e alle proprietà di sicurezza documentate.

2. Metodo di ispezione

L'analisi è stata organizzata in 20 punti di controllo. Per ogni punto sono stati valutati l'aspetto osservato, le evidenze presenti nel codice, eventuali criticità e un esito sintetico.

La classificazione dell'esito usa tre livelli:

- Soddisfatto: il codice rispetta il criterio considerato.
- Parzialmente soddisfatto: il criterio è rispettato nel nucleo principale, ma sono presenti miglioramenti consigliati o limiti localizzati.
- Da migliorare: il criterio presenta criticità significative rispetto all'obiettivo dell'ispezione.

3. Riepilogo dei 20 punti di controllo

N.	Punto di controllo	Criterio	Esito
1	Obiettivo dell'analisi	Definizione dello scopo dell'ispezione	Soddisfatto
2	Struttura del programma	Organizzazione in classi, package e responsabilità	Soddisfatto
3	Leggibilità del codice	Nomi, chiarezza, formattazione e comprensibilità	Soddisfatto
4	Correttezza logica	Coerenza di condizioni, cicli, transizioni e flusso	Soddisfatto
5	Precondizioni	Validità degli input e delle condizioni di chiamata	Soddisfatto
6	Postcondizioni	Coerenza dello stato dopo l'esecuzione dei metodi	Soddisfatto
7	Invarianti	Proprietà sempre vere sullo stato dell'ascensore	Soddisfatto
8	Casi limite	Piani estremi, input invalidi, assenza richieste, anomalie	Soddisfatto

9	Gestione errori	Null, guasto, overload, input non validi, errori HTTP	Parzialmente soddisfatto
10	Sicurezza	Movimento sicuro, protezione stati anomali e validazione input	Soddisfatto
11	Dipendenze	Uso di librerie e componenti esterni	Parzialmente soddisfatto
12	Duplicazione	Ripetizioni e possibilità di estrazione di codice comune	Parzialmente soddisfatto
13	Manutenibilità	Facilità di modifica, separazione e scalabilità	Parzialmente soddisfatto
14	Documentazione	Coerenza tra codice, JML e documenti descrittivi	Soddisfatto
15	Stile di programmazione	Convenzioni, ordine e uniformità	Parzialmente soddisfatto
16	Prestazioni	Possibili inefficienze rilevabili staticamente	Soddisfatto
17	Gestione risorse	Stream, server, driver, thread e sincronizzazione	Parzialmente soddisfatto
18	Testabilità	Isolabilità del nucleo e copertura tramite test	Soddisfatto
19	Codice morto/irraggiungibile	Metodi, rami o variabili poco utili	Parzialmente soddisfatto
20	Esito finale	Valutazione complessiva dell'ispezione	Soddisfatto

4. Analisi dettagliata punto per punto

1. Obiettivo dell'analisi

Esito: Soddisfatto

Osservazioni:

- L'obiettivo dell'ispezione è chiaro: verificare staticamente la qualità e la correttezza del codice Java/JML rispetto al comportamento atteso del sistema ascensore.
- Il codice viene analizzato come implementazione eseguibile del nucleo modellato formalmente, con particolare attenzione a invarianti, contratti JML, sicurezza del movimento, gestione degli errori e coerenza con i requisiti.
- L'analisi statica è correttamente distinta dai test JUnit, dai test Selenium e dalla validazione del modello ASM. Questi strumenti sono complementari, non sostitutivi dell'ispezione manuale.

2. Struttura generale del programma

Esito: Soddisfatto

Osservazioni:

- La struttura è organizzata in package con responsabilità distinte: progetto contiene il nucleo logico, web contiene l'interfaccia dimostrativa e test contiene la verifica automatica.
- La classe Ascensore incapsula lo stato e le operazioni fondamentali; ControlloreAscensore coordina l'ordine delle operazioni; InputAscensore raccoglie gli input esterni; le enum rappresentano domini finiti e riducono il rischio di valori non ammessi.
- La separazione tra stato, input e coordinamento rende il progetto leggibile e coerente con la trasformazione da modello ASM a implementazione Java.

Criticità o miglioramenti consigliati:

- La classe AscensoreHttpServer contiene sia logica HTTP, generazione HTML, CSS e JavaScript: questo rende il file molto lungo e meno modulare rispetto al nucleo logico.

3. Leggibilità del codice

Esito: Soddisfatto

Osservazioni:

- I nomi delle classi e dei metodi sono descrittivi e in italiano, ad esempio aggiungiRichiestaInterna, gestisciSovraccarico, gestisciTimerGuasto, scegliDirezione e muoviDiUnPiano.
- Le enumerazioni migliorano la leggibilità degli stati rispetto all'uso di costanti numeriche o stringhe libere.
- La formattazione generale è chiara e le sezioni dei test sono commentate per area funzionale.

Criticità o miglioramenti consigliati:

- La costruzione della pagina web tramite lunghe sequenze di html.append(...) riduce la leggibilità della parte web. Per un progetto più esteso sarebbe preferibile separare HTML, CSS e JavaScript in file dedicati.

4. Correttezza logica

Esito: Soddisfatto

Osservazioni:

- Il flusso del controllore è ordinato: acquisizione richieste, gestione persone, gestione errori e comportamento normale. Questo ordine è coerente con l'idea di passo logico del sistema.
- Il movimento avviene solo dopo la scelta della direzione e solo se porte chiuse e nessun errore attivo.
- Il servizio del piano corrente apre le porte, ferma la cabina, annulla la direzione e rimuove la richiesta associata al piano.
- La gestione del guasto e del sovraccarico è logicamente separata: OVERLOAD consente l'uscita delle persone, GUASTO blocca l'acquisizione di nuove richieste.

Criticità o miglioramenti consigliati:

- La parte visuale delle richieste nel server web è mantenuta separatamente dall'array richiesteAttive del nucleo. Questo può generare disallineamenti se la richiesta visuale viene registrata ma il controllore la ignora, ad esempio per chiamata salita al piano massimo o chiamata discesa al piano minimo in simulazione automatica.

5. Verifica delle precondizioni

Esito: Soddisfatto

Osservazioni:

- I metodi principali della classe Ascensore sono protetti da precondizioni JML: validità del piano, assenza di guasto, porte aperte o chiuse a seconda dell'operazione, stato cabina compatibile.
- Il metodo indiceDelPiano richiede pianoValido(piano), evitando accessi fuori dai limiti dell'array.
- Il controllore applica controlli difensivi sugli input prima di invocare metodi con precondizioni più restrittive, ad esempio ignora richieste non valide e valori negativi per persone entrate o uscite.
- Il costruttore di ControlloreAscensore rifiuta un oggetto Ascensore nullo e eseguiPasso rifiuta un InputAscensore nullo.

6. Verifica delle postcondizioni

Esito: Soddisfatto

Osservazioni:

- Le postcondizioni JML descrivono gli effetti dei metodi più importanti: aggiunta di richieste, aggiornamento del numero di persone, ingresso e uscita da overload, attivazione del guasto, decremento del timer, servizio del piano e movimento.
- Il costruttore garantisce lo stato iniziale atteso: piano 0, timer 0, nessun errore, porte chiuse e direzione nulla.
- Le postcondizioni del movimento assicurano che il piano corrente resti nei limiti e che agli estremi la cabina si fermi annullando la direzione.

Criticità o miglioramenti consigliati:

- Alcuni metodi hanno postcondizioni essenziali ma non completamente esaustive. Ad esempio scegliDirezione specifica alcuni casi principali, ma non descrive in modo completo tutti i rami decisionali possibili.

7. Verifica degli invarianti

Esito: Soddisfatto

Osservazioni:

- La classe Ascensore definisce invarianti JML significativi sullo stato numerico, sulla non nullità degli stati, sulla dimensione dell'array delle richieste e sulla sicurezza del movimento.
- Gli invarianti vietano configurazioni pericolose come cabina in movimento con porte aperte, movimento durante guasto, timer positivo fuori dallo stato GUASTO, stato GUASTO senza cabina bloccata o porte chiuse, e OVERLOAD senza porte aperte.
- Questi invarianti sono coerenti con le proprietà di safety del sistema e costituiscono il nucleo della verifica statica.

8. Controllo dei casi limite

Esito: Soddisfatto

Osservazioni:

- Il codice considera i piani estremi PIANO_MINIMO = -1 e PIANO_MASSIMO = 4.
- Le chiamate esterne sono filtrate: la salita non è ammessa al piano massimo e la discesa non è ammessa al piano minimo.
- Il movimento ai limiti impedisce di superare il piano minimo o massimo e annulla la direzione se non è possibile procedere.
- Sono gestiti casi di input nullo, valori negativi per persone entrate/uscite, richieste non valide, assenza di richieste, timer a zero e richieste al piano corrente.

9. Gestione degli errori

Esito: Parzialmente soddisfatto

Osservazioni:

- Il nucleo logico gestisce due errori principali: OVERLOAD e GUASTO.
- In stato GUASTO il sistema blocca la cabina, chiude le porte, annulla la direzione e usa un timer di ripristino.
- In stato OVERLOAD la cabina viene bloccata, le porte rimangono aperte e il sistema permette la riduzione del numero di persone per risolvere l'anomalia.
- La parte HTTP restituisce errore 405 per metodi non supportati e un JSON di errore in caso di RuntimeException.

Criticità o miglioramenti consigliati:

- La parte web tende a usare valori di default in caso di input non numerici. Questo evita crash, ma può nascondere errori dell'utente perché un valore non valido viene trasformato in 0.
- La gestione degli errori HTTP è essenziale ma non distingue molte tipologie di errore applicativo.

10. Controllo della sicurezza

Esito: Soddisfatto

Osservazioni:

- Dal punto di vista del modello applicativo, la sicurezza è ben rappresentata: niente movimento con porte aperte, niente movimento in guasto e blocco in sovraccarico.
- Le proprietà di sicurezza sono espresse sia tramite invarianti JML sia tramite controlli nel flusso del controllore.
- L'accesso diretto allo stato interno è impedito dall'uso di campi private e da metodi pubblici controllati.

Criticità o miglioramenti consigliati:

- La sicurezza informatica dell'interfaccia web è volutamente limitata perché si tratta di una demo locale: non sono presenti autenticazione, autorizzazione, CSRF protection o hardening HTTP. Questo è accettabile per un simulatore didattico locale, ma andrebbe rivisto in caso di esposizione reale.

11. Analisi delle dipendenze

Esito: Parzialmente soddisfatto

Osservazioni:

- Il nucleo logico dipende quasi solo da Java standard e dalle annotazioni JML.
- La parte web usa com.sun.net.httpserver.HttpServer, sufficiente per una dimostrazione leggera senza introdurre framework pesanti.
- I test usano JUnit e Selenium, adeguati rispettivamente per test unitari e test di integrazione web.

Criticità o miglioramenti consigliati:

- L'uso di com.sun.net.httpserver è adatto a scopi dimostrativi, ma non è pensato come base robusta per un'applicazione web reale.
- La presenza contemporanea di JUnit 4 nei test Selenium e JUnit 5 nei test del controllore può creare disomogeneità nella configurazione del progetto.

12. Controllo della duplicazione del codice

Esito: Parzialmente soddisfatto

Osservazioni:

- Nel nucleo logico la duplicazione è contenuta: i metodi sono specifici e il comportamento comune è in gran parte centralizzato.
- La conversione piano-indice è incapsulata in indiceDelPiano nella classe Ascensore e replicata nella parte web solo per esigenze visuali.

Criticità o miglioramenti consigliati:

- La validazione dei piani e la conversione piano-indice sono duplicate tra Ascensore e AscensoreHttpServer.
- La costruzione HTML contiene molte stringhe ripetute e logica visuale accorpata. Sarebbe utile estrarre componenti o template.
- Le chiamate manuali richiestaInterna, chiamataSalita e chiamataDiscesa hanno struttura molto simile e potrebbero essere generalizzate lato JavaScript.

13. Verifica della manutenibilità

Esito: Parzialmente soddisfatto

Osservazioni:

- Il nucleo logico è manutenibile perché le responsabilità sono separate: stato, input e coordinamento sono in classi distinte.
- L'uso di costanti per piani, capacità e timer facilita modifiche controllate.
- La presenza di test automatici rende più sicure eventuali modifiche future.

Criticità o miglioramenti consigliati:

- AscensoreHttpServer è molto grande e concentra server, routing, generazione pagina, stile, script, simulazione automatica e serializzazione JSON. Questo riduce la manutenibilità della parte web.
- L'aggiunta di nuovi stati o nuove regole richiederebbe modifiche coordinate in Ascensore, ControlloreAscensore, server web, test e documentazione.

14. Controllo della documentazione

Esito: Soddisfatto

Osservazioni:

- La documentazione caricata descrive requisiti, proprietà, implementazione Java/JML, validazione ASM, scenari AVALLA, ATGT, AsmetaSMV, JUnit e Selenium.
- La documentazione è coerente con il codice: i requisiti di acquisizione richieste, movimento sicuro, gestione guasto, overload, ripristino e conservazione richieste trovano corrispondenza nell'implementazione.
- Le annotazioni JML fungono da documentazione formale interna al codice e rendono esplicite le proprietà di sicurezza più importanti.

15. Verifica dello stile di programmazione

Esito: Parzialmente soddisfatto

Osservazioni:

- Il codice del nucleo usa uno stile chiaro, nomi espressivi e metodi relativamente brevi.
- Le enum sono usate correttamente per modellare stati discreti.
- La scelta di usare private per gli attributi e getter pure per l'accesso allo stato è coerente con un buon incapsulamento.

Criticità o miglioramenti consigliati:

- Sono presenti alcune differenze di indentazione, ad esempio tab in alcuni punti e spazi in altri.
- La parte web usa molte stringhe concatenate, rendendo lo stile meno pulito rispetto al resto del progetto.
- Nei test Selenium sono presenti Thread.sleep, soluzione semplice ma fragile rispetto ad attese esplicite.

16. Analisi preliminare delle prestazioni

Esito: Soddisfatto

Osservazioni:

- La complessità del nucleo è molto bassa: la ricerca delle richieste scorre al massimo NUMERO_PIANI elementi, quindi il costo è trascurabile nel dominio attuale.
- La simulazione automatica usa un intervallo di 1000 ms, compatibile con una demo e non intensivo per CPU.
- La serializzazione JSON e la generazione HTML sono semplici e gestibili per un singolo utente locale.

Criticità o miglioramenti consigliati:

- Se il numero di piani crescesse molto, la scansione lineare resterebbe probabilmente accettabile, ma si potrebbe valutare una struttura dati più specializzata. Per il progetto attuale non è necessario.

17. Controllo della gestione delle risorse

Esito: Parzialmente soddisfatto

Osservazioni:

- Gli stream HTTP vengono chiusi con try-with-resources nei metodi di risposta e nel test Selenium per l'invio del reset.
- Il server HTTP può essere avviato e fermato tramite avvia e ferma.
- Il driver Selenium viene chiuso in tearDown tramite driver.quit().
- La simulazione automatica usa un flag volatile e sincronizzazione sul server per proteggere lo stato condiviso.

Criticità o miglioramenti consigliati:

- fermaSimulazioneAutomatica interrompe il thread ma non esegue join; per una chiusura più rigorosa si potrebbe attendere la terminazione del thread.
- La porta 8080 è fissa: in contesti di test paralleli o ambienti CI complessi potrebbe essere già occupata.

18. Analisi della testabilità

Esito: Soddisfatto

Osservazioni:

- Il nucleo logico è altamente testabile perché Ascensore e ControlloreAscensore possono essere istanziati senza server web e senza dipendenze esterne.
- I test JUnit coprono stato iniziale, input validi e invalidi, movimento, richieste, porte, persone, overload, guasto, timer, ripristino e scelta direzione.
- I test Selenium verificano l'integrazione tra pagina, server e nucleo logico.
- La separazione tra test unitari e test di integrazione è corretta.

19. Individuazione di codice morto o irraggiungibile

Esito: Parzialmente soddisfatto

Osservazioni:

- Nel nucleo logico non emergono metodi chiaramente inutilizzati: i metodi pubblici di Ascensore sono usati dal controllore, dai test o dalla parte web.
- I rami principali del controllore risultano coperti da test significativi.

Criticità o miglioramenti consigliati:

- Alcuni rami della parte web e della simulazione casuale sono più difficili da verificare staticamente e potrebbero non essere completamente coperti dai test.
- La gestione degli input non numerici viene ricondotta a valori di default; ciò rende alcuni casi di errore non espliciti e quindi meno osservabili.

20. Esito finale dell'ispezione

Esito: Soddisfatto

Osservazioni:

- L'ispezione statica conferma che il progetto è complessivamente coerente, leggibile e ben organizzato nella parte centrale.
- Il nucleo Java/JML soddisfa le principali proprietà attese: validità dei piani, non negatività del numero di persone, assenza di stati nulli, movimento solo con porte chiuse, blocco in guasto, blocco in overload e ripristino tramite timer.
- Il controllore applica un flusso logico chiaro e coerente con i requisiti del sistema.
- Le criticità non compromettono il comportamento principale dell'ascensore, ma riguardano soprattutto modularità, robustezza e mantenibilità della parte web/dimostrativa.

5. Criticità individuate e miglioramenti consigliati

Priorità	Area	Criticità	Miglioramento consigliato
Media	Interfaccia web	La classe AscensoreHttpServer concentra server, HTML, CSS, JavaScript, simulazione e JSON.	Separare pagina HTML, CSS e JavaScript in file dedicati o introdurre un piccolo livello di template.
Media	Richieste visuali	Le richieste visuali sono gestite separatamente dalle richieste logiche e possono disallinearsi in casi limite.	Derivare la visualizzazione direttamente dallo stato logico oppure registrare la richiesta visuale solo dopo conferma dell'acquisizione da parte del controllore.
Bassa	Input web	Un input non numerico viene trasformato in valore 0.	Restituire un messaggio di errore esplicito oppure validare lato client e lato server.
Bassa	Test Selenium	Sono usati Thread.sleep per attendere aggiornamenti dell'interfaccia.	Usare WebDriverWait con condizioni esplicite sugli elementi attesi.
Bassa	Thread simulazione	Il thread viene interrotto ma non atteso con join.	Aggiungere join con timeout in fase di arresto della simulazione.
Bassa	Stile	Sono presenti piccole disomogeneità di indentazione.	Applicare un formatter Java uniforme e una convenzione di stile condivisa.

6. Punti di forza rilevati

- Separazione efficace tra stato dell'ascensore, input e logica di coordinamento.
- Uso di enum per rappresentare domini finiti e ridurre stati non ammessi.
- Presenza di invarianti JML significativi sulle proprietà di sicurezza.
- Gestione esplicita di guasto, sovraccarico e ripristino.
- Copertura dei principali casi limite tramite test JUnit.
- Test Selenium per verificare il collegamento tra interfaccia web e nucleo logico.
- Documentazione coerente con requisiti, implementazione e validazione del modello.

7. Conclusione

L'analisi statica del codice evidenzia un progetto complessivamente solido. Il nucleo logico dell'ascensore è chiaro, incapsulato e coerente con i requisiti funzionali e con le proprietà di sicurezza definite. La presenza di contratti JML rafforza la specifica formale del comportamento e permette di rendere esplicite le condizioni che devono rimanere sempre valide.

L'ispezione conferma in particolare che il sistema impedisce il movimento in condizioni non sicure, mantiene il piano corrente nei limiti ammessi, gestisce correttamente guasto e sovraccarico, conserva le richieste già acquisite durante le anomalie e consente il ripristino dopo il timeout del guasto.

Le principali aree di miglioramento riguardano la parte web dimostrativa, che potrebbe essere resa più modulare e più robusta, e alcuni aspetti di test e gestione risorse. Tali criticità sono localizzate e non compromettono la correttezza del nucleo logico. Nel complesso, l'esito dell'ispezione è positivo.